```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split

# 1. Load Data & Handle Missing Values
print("Handling Missing Values")

df = pd.read_csv("Dataset_Day9.csv")
data = df.copy()

missing_cols = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
for col in missing_cols:
    data[col] = data[col].replace(0, np.nan)
    median_val = data[col].median()
    print(f"Replaced 0s in '{col}' with median: {median_val}")
    data[col] = data[col].fillna(median_val)

# 2. Remove Outliers using Z-score

features = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
data_z = data[features]
z_scores = (data_z - data_z.mean()) / data_z.std()
outliers = (np.abs(z_scores) > 3).any(axis=1)
outlier_count = outliers.sum()
data_cleaned = data[~outliers]

print(f"Outliers removed: {outlier_count}")
print(f"Remaining rows after outlier removal: {data_cleaned.shape[0]}\n")
```

```
(myenv) flora_04@Flora:/mnt/d/python/day9$ python3 solution.py
Handling Missing Values
Replaced 0s in 'Glucose' with median: 117.0
Replaced 0s in 'BloodPressure' with median: 72.0
Replaced 0s in 'BMI' with median: 32.3
Replaced 0s in 'DiabetesPedigreeFunction' with median: 0.3725
Replaced 0s in 'Age' with median: 29.0
Outliers removed: 26
Remaining rows after outlier removal: 742
```

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
from itertools import product

# Load dataset (replace with your actual path if needed)
data = pd.read_csv('Dataset_Day10.csv')
print("Dataset Loaded Successfully.\n")

# Split data into features and target
X = data.drop('Outcome', axis=1)
y = data['Outcome']
print("Feature and Target variables split. Target variable: 'Outcome'\n")

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data split into 80% training and 20% testing sets.\n")

# Train Decision Tree Classifier with default parameters
clf_default = DecisionTreeClassifier(random_state=42)
clf_default.fit(X_train, y_train)
y_pred_default = clf_default.predict(X_test)
print("Default Decision Tree model trained.\n")

# Evaluate default model performance
acc = accuracy_score(y_test, y_pred_default)
prec = precision_score(y_test, y_pred_default)
rec = recall_score(y_test, y_pred_default)
f1 = f1_score(y_test, y_pred_default)

print("Default Decision Tree Performance Metrics:")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1 Score : {f1:.4f}\n")
```

```python
# Tune hyperparameters and record performance
max_leaf_nodes_range = range(2, 20, 2)
max_depth_range = range(2, 10)

results = []
best_f1 = 0
best_params = (None, None)

for leaf_nodes, depth in product(max_leaf_nodes_range, max_depth_range):
    clf = DecisionTreeClassifier(
        criterion='entropy',
        max_leaf_nodes=leaf_nodes,
        max_depth=depth,
        random_state=42
    )
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append((leaf_nodes, depth, prec, rec, f1))

    if f1 > best_f1:
        best_f1 = f1
        best_params = (leaf_nodes, depth)

print("Hyperparameter tuning completed.")
print(f"Best F1-score: {best_f1:.4f}")
print(f"Best Parameters: max_leaf_nodes = {best_params[0]}, max_depth = {best_params[1]}\n")
```

```python
# Plot Precision & Recall vs max_leaf_nodes grouped by max_depth
results_df = pd.DataFrame(results, columns=['max_leaf_nodes', 'max_depth', 'precision', 'recall', 'f1'])

plt.figure(figsize=(12, 6))
for depth in max_depth_range:
    subset = results_df[results_df['max_depth'] == depth]
    plt.plot(subset['max_leaf_nodes'], subset['precision'], label=f'Precision (depth={depth})', linestyle='--')
    plt.plot(subset['max_leaf_nodes'], subset['recall'], label=f'Recall (depth={depth})')

plt.xlabel('max_leaf_nodes')
plt.ylabel('Score')
plt.title('Precision & Recall vs max_leaf_nodes (grouped by max_depth)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print("Precision and Recall plot displayed successfully.\n")
```

```
Data split into 80% training and 20% testing sets.

Default Decision Tree model trained.

Default Decision Tree Performance Metrics:
Accuracy : 0.7273
Precision: 0.6140
Recall   : 0.6364
F1 Score : 0.6250

Hyperparameter tuning completed.
Best F1-score: 0.7069
Best Parameters: max_leaf_nodes = 12, max_depth = 5

Precision and Recall plot displayed successfully.
```